



Unified MCP Framework for Context-Aware AI Agents in Software Development

Harsh Kumar (221031040) | Naviya Tickoo (221030050) | Anjali Panwar (221030113) | Om Shree (221030410) | B.Tech CSE 4th Yr.

Under the Supervision of Dr.Aman Sharma Department of Computer Science & Engineering of Information Technology

INTRODUCTION

Modern software development is inherently fragmented. Developers constantly switch between local IDEs, web browsers, GitHub repositories and cloud dashboards - each context switch breaking focus and losing state. Existing AI assistants work within a single environment and cannot follow the developer across tools[1][2][3]. The Model Context Protocol (MCP), introduced in 2024, provides a structured, secure standard for AI agents to discover and interact with external tools. This project builds on MCP to create the Unified MCP Framework - a multi-agent system that brings filesystem operations, web browsing, GitHub access, code execution, weather and system diagnostics under a single natural language interface.

PROBLEM STATEMENT

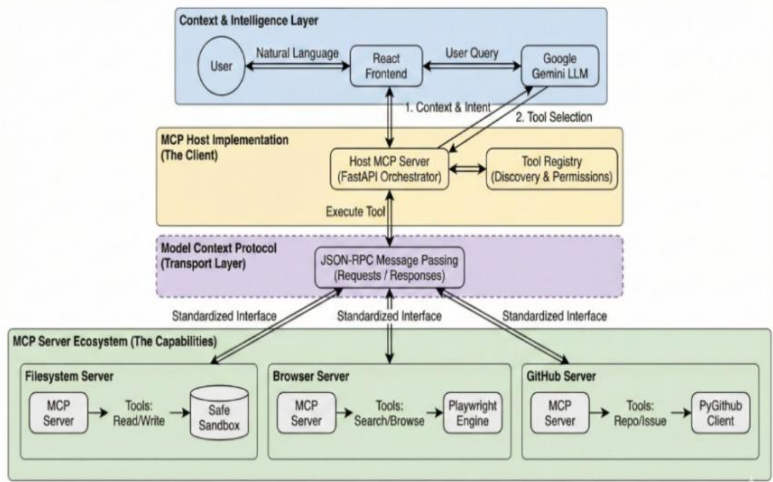
- Developers lose productivity due to switching between IDEs, browsers, and GitHub.
- Current AI assistants lack secure coordination across tools.
- A Unified MCP Framework is proposed to securely integrate Desktop, Browser, and GitHub MCP servers.
- The system enables a seamless, secure, and context-aware development workflow.

OBJECTIVES

The project aims to design and implement three modular MCP servers including Desktop, Browser, and GitHub ensuring seamless communication, interoperability, and scalability across developer environments. A central Host MCP Server is then developed to orchestrate all inter-server communication through a standardized protocol layer for multi-agent coordination. Finally, a full-stack agent-driven platform is built with a modern frontend, backend, and database enabling developers to interact with AI agents for coding, debugging, and workflow automation.

METHODOLOGY

The system starts with a **user query** entered through the developer interface. The query is processed using **intent parsing**, **context extraction**, and token normalization to improve routing accuracy. The processed request is handled by the **Host MCP Server**, which coordinates **Desktop MCP**, **Browser MCP**, and **GitHub MCP** for tool selection and task execution based on the user’s needs. Finally, the results along with relevant code suggestions or fetched data are returned to the frontend, enabling seamless and real time developer assistance

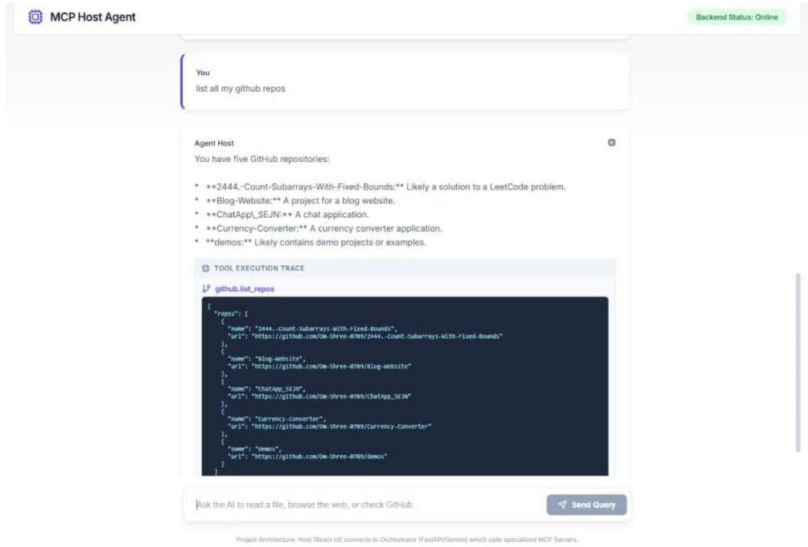


IMPLEMENTATION

The system is built on five coordinated components. The **MCP Core** defines the MCPTool schema and an abstract base class (IMCPEXternalServer); a run_tool wrapper handles validation and sync/async switching. The **Host MCP Server** orchestrates all queries through an LLM-driven pipeline — intent → tool selection → execution → summarization. The **Filesystem MCP** enforces a strict sandbox with a _get_safe_path gatekeeper blocking all path traversal (..) attacks. The **Browser MCP** enables read-only web research via Playwright, blocking unsafe URLs to prevent SSRF attacks and reusing a shared browser instance for speed. The **GitHub MCP** accesses repositories via API, handling authentication and base64 decoding transparently for the LLM.

RESULTS

The experiment showed that MCP improved LLM performance and enabled efficient backend coordination. **Filesystem MCP**, **Browser MCP**, and **GitHub MCP** confirmed the reliability of the modular MCP architecture.



CONCLUSION AND FUTURE SCOPE

The project demonstrated the effectiveness of MCP in improving LLM performance and developer assistance.

Future Scope:

- Expand the MCP ecosystem and agent capabilities
- Enable cloud deployment and IDE/CI-CD integration
- Strengthen security architecture and tool governance

REFERENCES

- [1]H. Song et al., “Beyond the Protocol: Unveiling Attack Vectors in the Model Context Protocol Ecosystem,” arXiv.org, 2025. <https://arxiv.org/abs/2506.02040>
- [2]N. V. Sai and I. Habler, “Enterprise-Grade Security for the Model Context Protocol (MCP): Frameworks and Mitigation Strategies,” arXiv.org, 2025. <https://arxiv.org/abs/2504.08623>
- [3]N. Croce and T. South, “Trivial Trojans: How Minimal MCP Servers Enable Cross-Tool Exfiltration of Sensitive Data,” arXiv.org, 2025. <https://arxiv.org/abs/2507.19880>